

Problem Statements

1. HomeNest — Real Estate Listing Platform (React)

A real estate agency wants a website to showcase property listings and generate leads from prospective buyers/renters. Your team must design and develop the frontend prototype using **React** (with React Router), **component-based architecture**, and **Git/GitHub**, following clean coding and version control practices.

Tech Requirements

Requirement	Details
Framework	React (functional components + hooks: useState, useEffect)
Routing	React Router DOM — /, /properties, /contact, /about
Architecture	Component-driven structure (reusable components, no copy-pasted markup)
Styling	CSS Modules / styled-components / Tailwind (team's choice, kept consistent)
Forms	Controlled components with JS-based validation
Data Flow	Props for dynamic data (property cards, agent cards from arrays/JSON)

Pages to Build

1. Home (/) — Landing Page

- **Navbar** component with logo & NavLink menu (active-link styling)
- **Hero** component: headline, description, CTA button ("Browse Properties")
- **FeatureCard** component via .map() — minimum 4 cards (Verified Listings, Home Loans, Virtual Tours, Legal Assistance)
- **Benefits** section
- **Footer** component with contact/social info

2. Properties (/properties) — Listings Page

- Listings overview section
- **PropertyCard** list rendered dynamically from a data array/state (title, location, size, price)
- Filter/plans-style section (Buy / Rent / Lease)
- Client testimonial carousel or list (mapped over array)

- **FAQ accordion** built with useState to toggle open/close

3. Contact (/contact) — Site Visit / Enquiry Form

Controlled form (all inputs tied to state, validated on submit):

- Full Name
- Email
- Phone Number
- Preferred Visit Date
- Dropdown — Property Type (Apartment / Villa / Plot / Commercial)
- Radio Buttons — Purpose (Buy / Rent / Lease)
- Checkboxes — Amenities Interested In (multiple)
- Text Area — Message/Query
- Submit & Reset buttons

Validation rules:

- No HTML5-only validation — use JS state-driven validation
- Show error messages conditionally
- On successful submit, show a success message component (conditional rendering) instead of actually sending data

4. About (/about) — About Agency & Team

- Mission/vision section
- **AgentCard** component via .map() over an agent data array (photo, name, role, bio)
- Company story timeline

Deliverables

- Fully responsive (flexbox/grid + media queries)
- Clean component folder structure (/components, /pages, /data)
- State lifted appropriately (discuss if Context API is warranted)
- Git repo with meaningful, incremental commits (feature-by-feature)
- README with setup/run instructions (npm install, npm start)

2. LearnHub — Online Course Platform (React)

Tech Requirements

Requirement	Details
Framework	React (functional components + hooks: useState, useEffect)
Routing	React Router DOM — /, /courses, /contact, /about
Architecture	Component-driven structure (reusable components, no copy-pasted markup)
Styling	CSS Modules / styled-components / Tailwind (team's choice, kept consistent)
Forms	Controlled components with JS-based validation
Data Flow	Props for dynamic data (course cards, team cards from arrays/JSON)

Pages to Build

1. Home (/) — Landing Page

- **Navbar** component with logo & NavLink menu (active-link styling)
- **Hero** component: headline, description, CTA button ("Explore Courses")
- **FeatureCard** component rendered via .map() — minimum 4 cards (Live Classes, Certification, Mentorship, Flexible Schedule)
- **Benefits** section
- **Footer** component with contact/social info

2. Courses (/courses) — Course Showcase Page

- Course overview section
- **CourseCard** list rendered dynamically from a data array/state (title, instructor, duration, price)
- Pricing/plans section (Basic / Standard / Pro)
- Testimonial carousel or list (mapped over testimonials array)
- **FAQ accordion** built with useState to toggle open/close

3. Contact (/contact) — Enrollment / Contact Form

Controlled form (all inputs tied to state, validated on submit):

- Full Name

- Email
- Phone Number
- Preferred Start Date
- Dropdown — Course Category (Web Dev / Data Science / Design / Marketing)
- Radio Buttons — Mode (Online / Offline / Hybrid)
- Checkboxes — Interests (multiple course topics)
- Text Area — Message/Query
- Submit & Reset buttons

Validation rules:

- No HTML5-only validation — use JS state-driven validation
- Show error messages conditionally
- On successful submit, show a success message component (conditional rendering) instead of actually sending data

4. About (/about) — About Platform & Team

- Mission/vision section
- **TeamCard** component rendered via `.map()` over a team data array (photo, name, role, bio)
- Company story timeline

Deliverables

- Fully responsive (flexbox/grid + media queries)
- Clean component folder structure (`/components`, `/pages`, `/data`)
- State lifted appropriately (no unnecessary prop drilling — discuss if Context API is warranted)
- Git repo with meaningful, incremental commits (feature-by-feature, not one dump)
- README with setup/run instructions (`npm install`, `npm start`)

3. TravelNest — Travel & Tour Booking Platform (React)

A travel agency startup wants a website to showcase tour packages and destinations to attract travelers and generate bookings. Your team must design and develop the frontend prototype using **React** (with React Router), **component-based architecture**, and **Git/GitHub**, following clean coding and version control practices.

Tech Requirements

Requirement	Details
Framework	React (functional components + hooks: useState, useEffect)
Routing	React Router DOM — /, /packages, /contact, /about
Architecture	Component-driven structure (reusable components, no copy-pasted markup)
Styling	CSS Modules / styled-components / Tailwind (team's choice, kept consistent)
Forms	Controlled components with JS-based validation
Data Flow	Props for dynamic data (package cards, guide cards from arrays/JSON)

Pages to Build

1. Home (/) — Landing Page

- **Navbar** component with logo & NavLink menu (active-link styling)
- **Hero** component: headline, description, CTA button ("Explore Destinations")
- **FeatureCard** component via .map() — minimum 4 cards (Guided Tours, Visa Assistance, Custom Itineraries, 24/7 Support)
- **Benefits** section
- **Footer** component with contact/social info

2. Packages (/packages) — Tour Package Page

- Package overview section
- **PackageCard** list rendered dynamically from a data array/state (destination, duration, group size, price)
- Pricing/plans section (Budget / Standard / Luxury)
- Traveler testimonial carousel or list (mapped over array)
- **FAQ accordion** built with useState to toggle open/close

3. Contact (/contact) — Trip Enquiry Form

Controlled form (all inputs tied to state, validated on submit):

- Full Name
- Email
- Phone Number
- Preferred Travel Date
- Dropdown — Destination Type (Beach / Mountain / Heritage / International)
- Radio Buttons — Travel Style (Solo / Family / Group)
- Checkboxes — Interests (Adventure / Sightseeing / Food Tours / Wildlife)
- Text Area — Message/Query
- Submit & Reset buttons

Validation rules:

- No HTML5-only validation — use JS state-driven validation
- Show error messages conditionally
- On successful submit, show a success message component (conditional rendering) instead of actually sending data

4. About (/about) — About Agency & Team

- Mission/vision section
- **GuideCard** component via `.map()` over a guide data array (photo, name, expertise, bio)
- Agency story timeline

Deliverables

- Fully responsive (flexbox/grid + media queries)
- Clean component folder structure (`/components`, `/pages`, `/data`)
- State lifted appropriately (discuss if Context API is warranted)
- Git repo with meaningful, incremental commits (feature-by-feature)
- README with setup/run instructions (`npm install`, `npm start`)